

DecodeLabs Internship - Project 4

Image Text Recognition using OCR

1. Project Overview

This project was completed as part of the DecodeLabs AI Internship. The main purpose of the project is to apply a basic computer vision technique that allows a machine to read text from an image. In real life, many documents, receipts, forms, invoices, certificates, and scanned papers are stored as images. These images cannot be searched or edited directly unless the text inside them is extracted first.

To solve this problem, this project uses Optical Character Recognition, also known as OCR. OCR is a technique that helps the computer detect and convert text from an image into editable digital text. The project uses Python with OpenCV for image preprocessing and pytesseract for extracting text from the processed image.

The system takes an image as input, improves its quality through preprocessing steps, applies OCR, prints the extracted text, and saves the final results in an output folder.

2. Project Objective

The objective of this project is to build a simple image text recognition system using Python. The system should be able to read an image that contains text, process the image to make the text clearer, and then extract the text using an OCR engine.

The project aims to demonstrate the following:

- How to load an image using Python.
 - How to preprocess an image before applying OCR.
 - How to convert an image into grayscale.
 - How to reduce noise using Gaussian blur.
 - How to improve text visibility using adaptive thresholding.
 - How to extract text from an image using pytesseract.
 - How to save both the processed image and extracted text.
 - How to organize the project files in a clean and professional structure.
-

3. Problem Statement

Many types of information are stored in image format, such as scanned documents, screenshots, receipts, handwritten notes, and printed forms. The problem is that text inside an image is not directly editable or searchable. A user cannot copy the text from an image unless it is manually typed or extracted using OCR.

Manual typing is slow and can cause mistakes, especially when dealing with many images or long documents. Therefore, this project provides a simple automated solution that extracts text from an image using OCR.

The main problem solved by this project is:

How can we automatically extract readable text from an image using Python and computer vision techniques?

4. Project Type

This project belongs to the field of **Computer Vision**.

Computer Vision is a branch of Artificial Intelligence that allows computers to understand and process visual data such as images and videos. In this project, the computer is not detecting objects, but it is recognizing text from an image.

The specific task used in this project is:

Optical Character Recognition OCR

OCR is used to recognize characters, words, and sentences from image input and convert them into digital text.

5. Tools and Technologies Used

The project was implemented using the following tools and technologies:

Tool / Library	Purpose
Python	Main programming language used to build the project
OpenCV	Used for image loading and preprocessing
pytesseract	Python wrapper used to connect Python with Tesseract OCR
Tesseract OCR Engine	The OCR engine responsible for extracting text from images

Tool / Library	Purpose
Pillow	Image handling support library
Matplotlib	Used in the notebook to display images
Jupyter Notebook	Used to explain and test the project step by step
VS Code / Terminal	Used to run the Python file

6. Project Folder Structure

The project was organized in a clear structure to make it easy to understand and run.

```

Project_4/
├── main.py
├── notebook.ipynb
├── requirements.txt
├── README.md
├── sample_images/
│   └── sample_text.png
└── outputs/
    ├── processed_image.png
    └── extracted_text.txt

```

File Description

File / Folder	Description
main.py	The main Python script that runs the OCR pipeline
notebook.ipynb	A Jupyter Notebook that explains the project step by step
requirements.txt	Contains the required Python libraries
README.md	Project documentation and running instructions
sample_images	Folder that contains the input image
sample_text.png	The sample image used for text recognition
outputs	Folder that stores the final results
processed_image.png	The image after preprocessing
extracted_text.txt	The final extracted text from the image

7. Dataset Description

This project does not use a traditional machine learning dataset such as a CSV file or a large labeled dataset. Instead, it uses a sample image that contains readable text.

The input image is stored in the following folder:

`sample_images/sample_text.png`

The image contains printed English text related to the project. This image is used as the input for the OCR system. Since the project uses a pre-trained OCR engine, there is no need to manually collect or label a large dataset.

The project depends on an input image rather than a training dataset because the OCR model is already available through the Tesseract OCR engine.

8. Algorithm and Technique Used

The main technique used in this project is:

Optical Character Recognition OCR

OCR is a computer vision technique used to detect and extract text from images. It analyzes the shapes of characters in an image and converts them into editable text.

In this project, the OCR process is supported by image preprocessing. Preprocessing is important because OCR accuracy depends heavily on the quality of the input image. If the image is noisy, blurry, dark, or unclear, the OCR result may contain mistakes.

The project uses the following preprocessing techniques:

8.1 Grayscale Conversion

The original image is loaded in color format. A color image usually contains three channels: blue, green, and red. OCR does not always need color information, so the image is converted into grayscale.

Grayscale conversion reduces the image to one intensity channel. This makes processing simpler and helps the OCR system focus on the text shape rather than colors.

8.2 Gaussian Blur

Gaussian blur is applied to reduce small noise in the image. Noise can appear as small unwanted dots or rough edges. These small disturbances may affect the OCR result.

By applying blur, the image becomes smoother, and the OCR engine can focus better on the main text.

8.3 Adaptive Thresholding

Adaptive thresholding converts the grayscale image into a black-and-white image. This step helps separate the text from the background.

Instead of using one fixed threshold value for the whole image, adaptive thresholding calculates the threshold based on local regions of the image. This makes it useful when the lighting is not equal across the image.

8.4 OCR Text Extraction

After preprocessing, the image is passed to pytesseract. The pytesseract library sends the image to the Tesseract OCR engine, which extracts the text and returns it as a string.

The extracted text is then printed in the terminal and saved in a text file.

9. Project Methodology

The project was implemented through a simple pipeline.

Step 1: Import Required Libraries

The first step was importing the required Python libraries:

- `os`
- `cv2`
- `pytesseract`
- `matplotlib.pyplot`

The `os` library was used to work with folders and file paths. OpenCV was used for image processing. `pytesseract` was used for OCR. `Matplotlib` was used in the notebook to display images.

Step 2: Load the Input Image

The image was loaded from the `sample_images` folder using OpenCV.

The image path used in the project is:

```
sample_images/sample_text.png
```

A validation check was added to make sure the image exists. If the image path is wrong, the program shows an error message instead of continuing.

Step 3: Convert the Image to Grayscale

The image was converted from BGR format to grayscale using OpenCV. This step reduces image complexity and prepares the image for better OCR processing.

Step 4: Apply Gaussian Blur

Gaussian blur was applied to reduce noise in the grayscale image. This improves the quality of the image before applying thresholding.

Step 5: Apply Adaptive Thresholding

Adaptive thresholding was used to make the text more visible. The result is a processed black-and-white image where the text is clearer and easier to recognize.

Step 6: Extract Text Using pytesseract

The processed image was passed to the OCR engine using:

```
pytesseract.image_to_string(processed_image)
```

This function extracts the readable text from the processed image.

Step 7: Save the Outputs

The project saves two output files:

```
outputs/processed_image.png  
outputs/extracted_text.txt
```

The processed image is saved to show the result of preprocessing. The extracted text is saved in a text file so it can be reviewed or used later.

10. Main Code Explanation

The project code is divided into clear functions.

`preprocess_image` function

This function is responsible for preparing the image before OCR. It loads the image, converts it to grayscale, applies blur, and applies adaptive thresholding.

`extract_text` function

This function receives the processed image and extracts text from it using pytesseract.

`save_results` function

This function saves the processed image and the extracted text into the outputs folder.

main function

The main function controls the full program flow. It calls the preprocessing function, extracts the text, saves the outputs, and prints the result in the terminal.

11. Project Output

The expected output of the project is the extracted text from the input image.

Example output:

```
Extracted Text:  
DecodeLabs Internship - Project 4  
Image Text Recognition using OCR  
Student Name: Zeyad Mohamed  
Goal: Extract readable text from an image.
```

The project also saves the processed image and the extracted text in the outputs folder.

12. Screenshots Section

File Edit Selection View Go Run

Project_4

EXPLORER

OPEN EDITORS

Release Notes: 1.122.1

main.py

requirements.txt

notebook.ipynb

PROJECT_4

outputs

sample_images

sample_text.png

main.py

notebook.ipynb

README.md

requirements.txt

OUTLINE

TIMELINE

notebook.ipynb > M4 DecodeLabs Internship - Project 4 > M4 Step 6: Save the Outputs > os.makedirs('outputs', exist_ok=True)

Generate + Code + Markdown | Run All | Clear All Outputs | Outline

Python 3.12.10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

G:\intern ship at decode labs\Project_4>python main.py

Project 4: Image Text Recognition using OCR

Extracted Text:

This is a lot of 12 point text to test the ocr code and see if it works on all types of file format.

The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox.

Processed image saved to: outputs\processed_image.png
Extracted text saved to: outputs\extracted_text.txt

G:\intern ship at decode labs\Project_4

Tip: Try the Plan agent to research and plan before implementing changes.

+ notebook.ipynb • Cell 16

Describe what to build

Activate Windows

Go to Settings to activate Windows

Local Default Approvals

Spaces: 4

Cell 16 of 17

Go Live

11:57 PM 5/30/2026

File Edit Selection View Go Run

Project_4

EXPLORER

OPEN EDITORS

Release Notes: 1.122.1

main.py

requirements.txt

notebook.ipynb

PROJECT_4

outputs

sample_images

sample_text.png

main.py

notebook.ipynb

README.md

requirements.txt

OUTLINE

TIMELINE

notebook.ipynb > M4 DecodeLabs Internship - Project 4 > M4 Step 6: Save the Outputs > os.makedirs('outputs', exist_ok=True)

Generate + Code + Markdown | Run All | Clear All Outputs | Outline

Python 3.12.10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

The processed image and extracted text are saved in the outputs folder.

os.makedirs('outputs', exist_ok=True)

cv2.imwrite(os.path.join('outputs', 'processed_image.png'), processed)

with open(os.path.join('outputs', 'extracted_text.txt'), 'w', encoding='utf-8') as file:

file.write(extracted_text.strip())

print('Processed image saved to outputs/processed_image.png')

print('Extracted text saved to outputs/extracted_text.txt')

[25] ✓ 0.0s Python

Processed image saved to outputs/processed_image.png
Extracted text saved to outputs/extracted_text.txt

Tip: Try the Plan agent to research and plan before implementing changes.

+ notebook.ipynb • Cell 16

Describe what to build

Activate Windows

Go to Settings to activate Windows

Local Default Approvals

Spaces: 4

Cell 16 of 17

Go Live

11:56 PM 5/30/2026

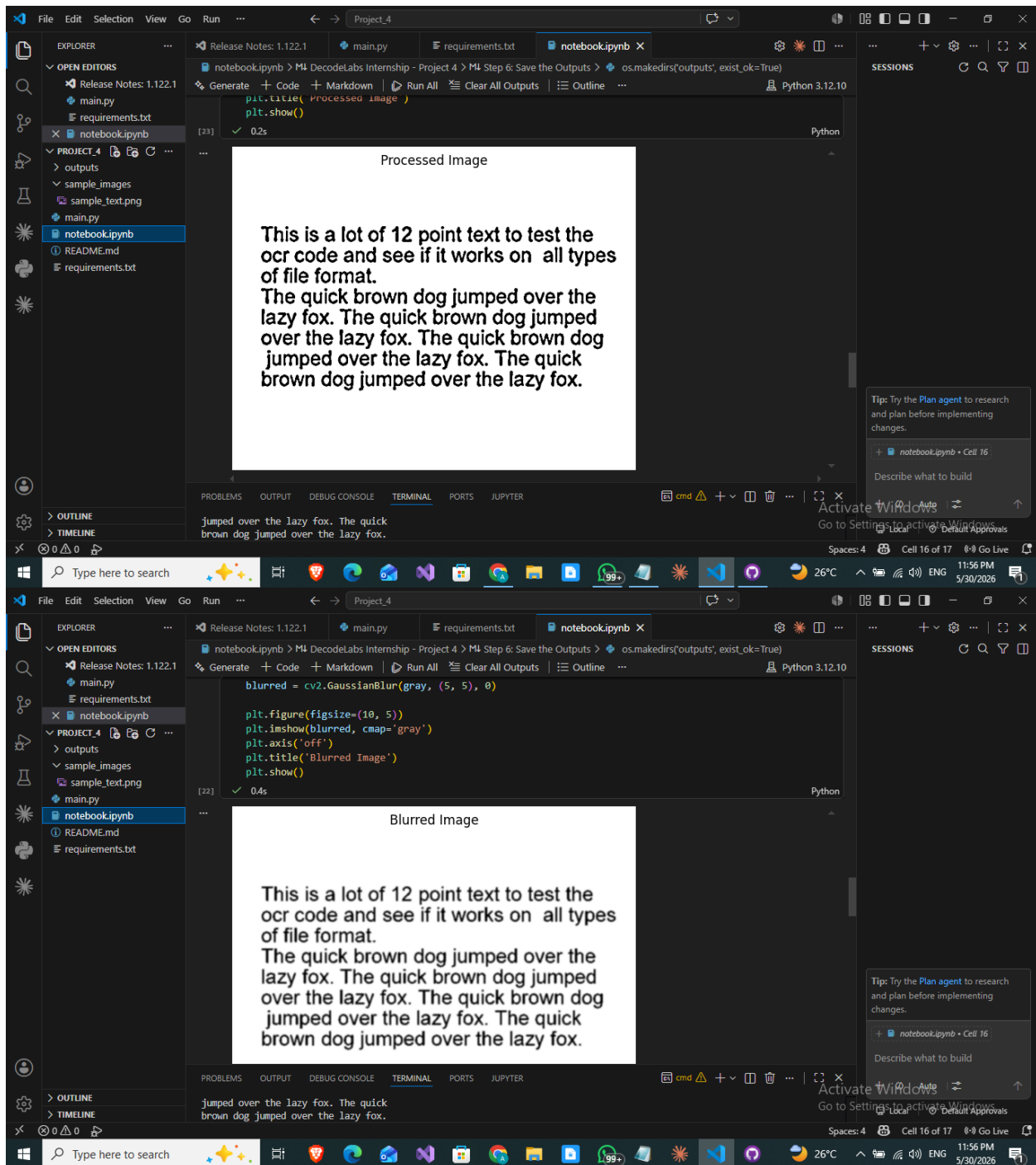
Conclusion

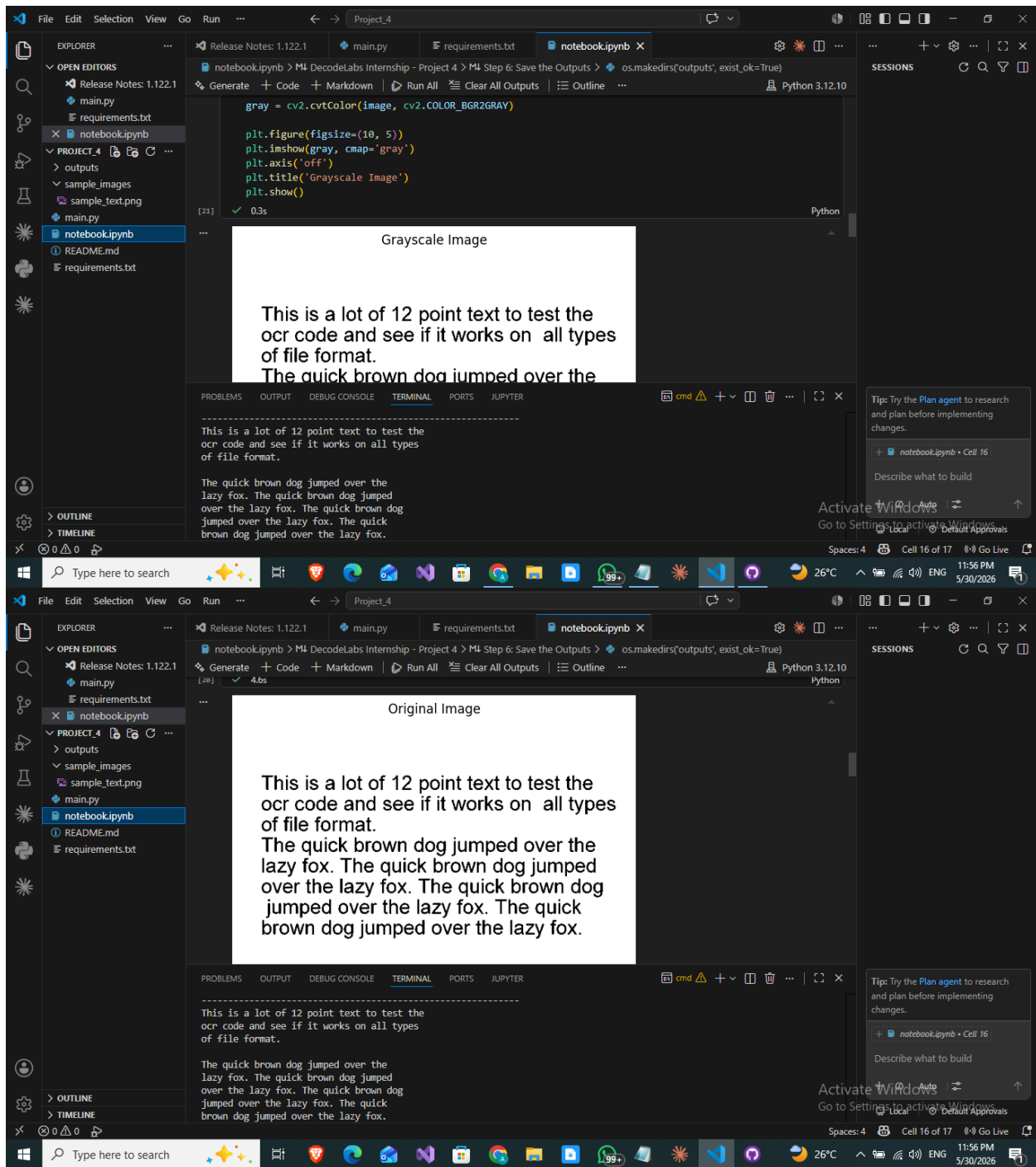
This project demonstrates a basic OCR pipeline. The system loads an image, applies preprocessing, extracts readable text using pytesseract, and saves the final results. This is a practical example of image/text recognition using available AI and computer vision libraries.

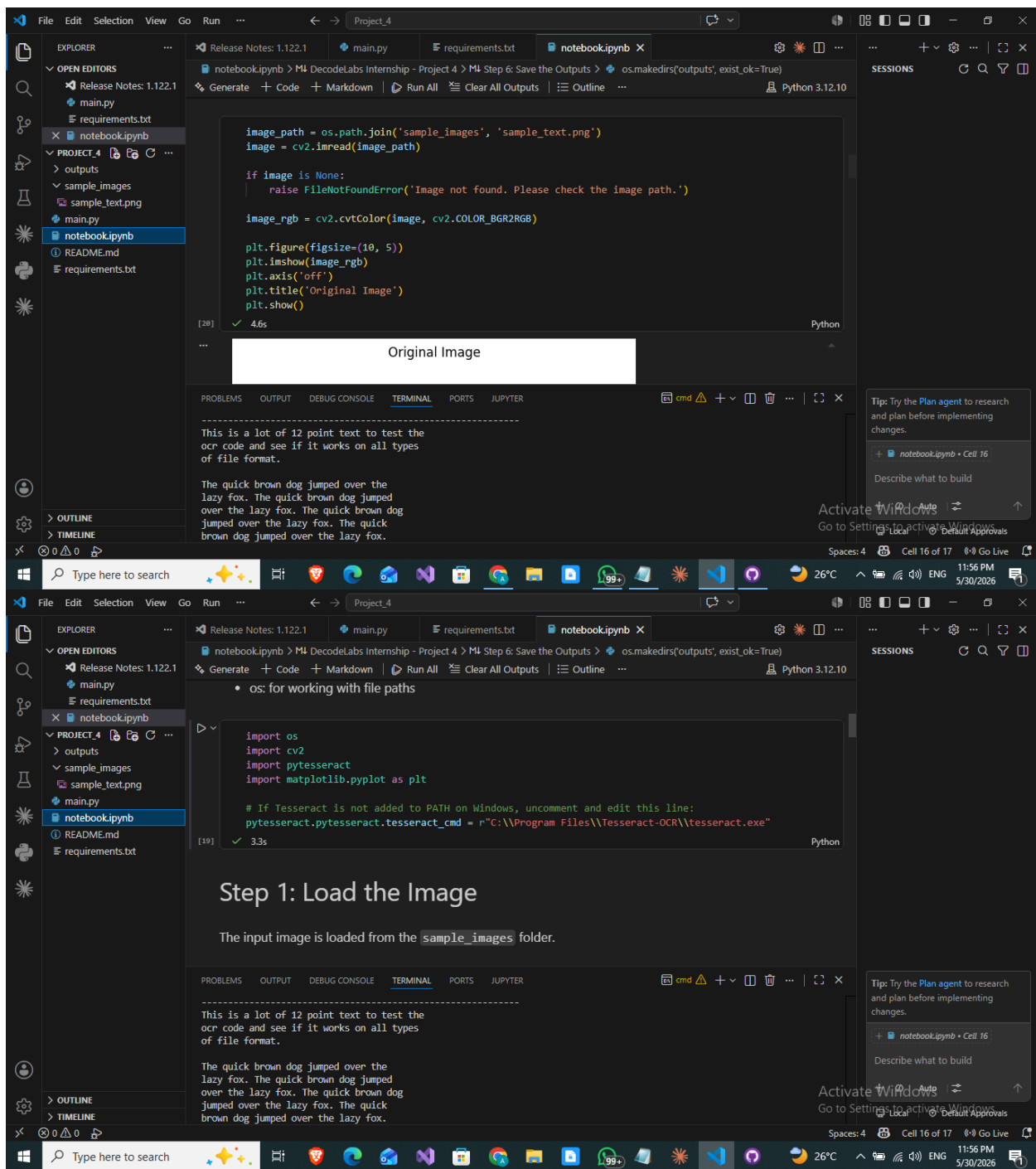
Generate + Code + Markdown

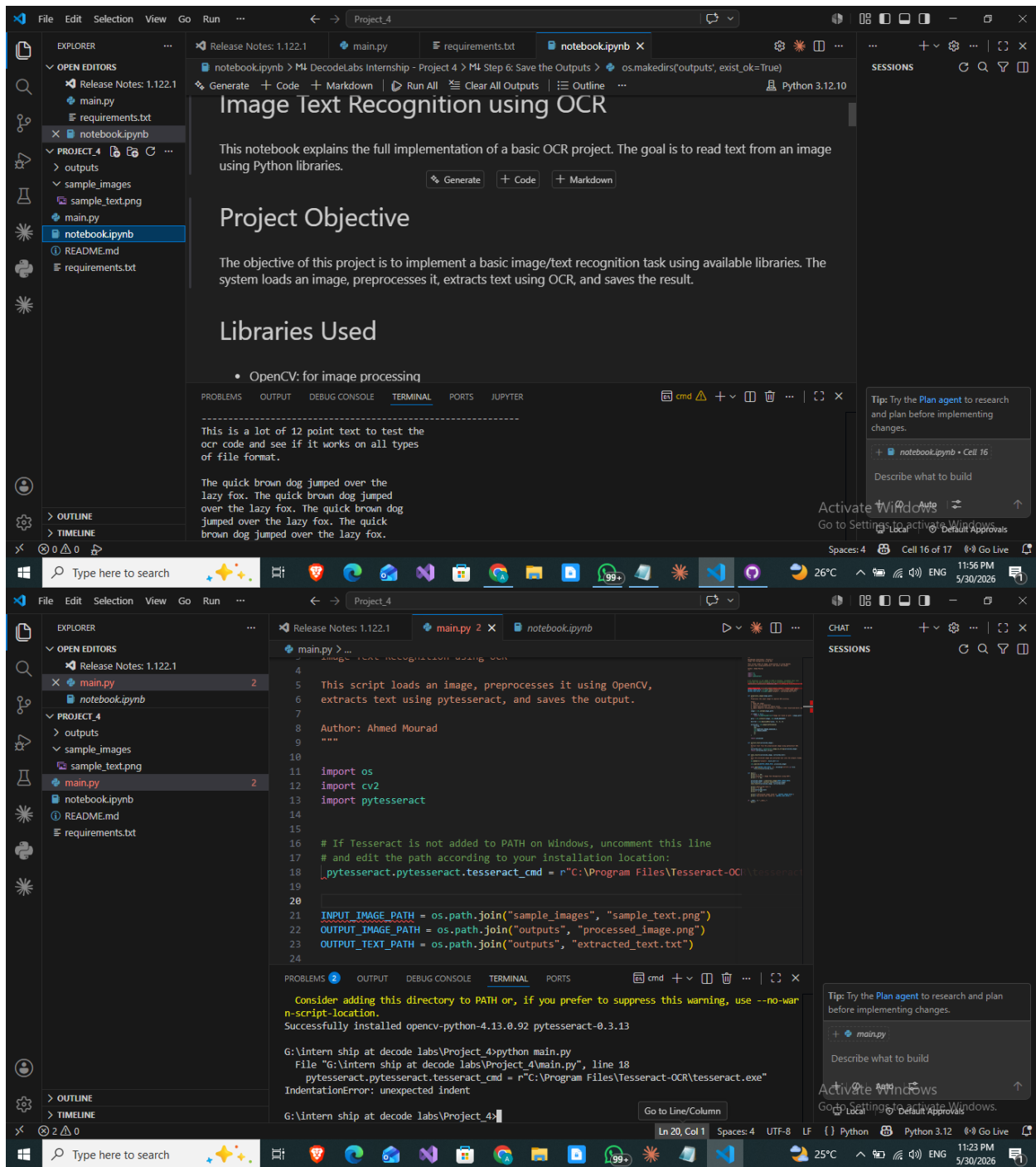
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

jumped over the lazy fox. The quick brown dog jumped over the lazy fox.









13. Challenges Faced

During the project, one important challenge was understanding the difference between pytesseract and the Tesseract OCR engine.

The Python package pytesseract is only a wrapper that allows Python to communicate with the OCR engine. However, it does not include the actual OCR engine itself. Therefore, Tesseract OCR must be installed separately on Windows.

Another challenge was making sure the path to tesseract.exe was correctly written in the Python file. Without the correct path, Python cannot find the OCR engine and the program gives an error.

The issue was solved by installing Tesseract OCR and adding the correct path in the code:

```
pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
```

Another challenge was that some notebook cells depend on variables created in previous cells. For example, the saving cell depends on processed and extracted_text. If the previous cells are not executed first, the saving cell will give an error. This was solved by running the notebook cells in the correct order.

14. What I Have Done in This Project

In this project, I completed the following tasks:

- Created a professional folder structure for the project.
 - Added a sample image for testing OCR.
 - Created a Python script named main.py.
 - Created a Jupyter Notebook to explain the project step by step.
 - Added image preprocessing using OpenCV.
 - Converted the image to grayscale.
 - Applied Gaussian blur to reduce noise.
 - Applied adaptive thresholding to improve text visibility.
 - Used pytesseract to extract text from the processed image.
 - Saved the processed image into the outputs folder.
 - Saved the extracted text into a text file.
 - Created a README file explaining the project.
 - Added a requirements file containing the needed libraries.
 - Tested the project using the terminal and notebook.
-

15. Results and Discussion

The project successfully demonstrates a basic OCR pipeline. The program can read an image, preprocess it, and extract text from it. The result shows that image preprocessing improves the quality of the image before OCR.

The project is simple but important because it introduces one of the most practical computer vision applications. OCR is widely used in document scanning, invoice reading, ID card recognition, banking systems, digital archiving, and automation.

The final output proves that the machine can process visual text and convert it into editable text.

16. Possible Enhancements

The project can be improved in several ways in the future.

16.1 Multiple Image Support

Instead of processing only one image, the system can be improved to process all images inside a folder automatically.

16.2 PDF Text Extraction

The project can be extended to extract text from scanned PDF files by converting each PDF page into an image and applying OCR.

16.3 Arabic OCR Support

The current project focuses mainly on English text. It can be improved by adding support for Arabic OCR using the Arabic language package in Tesseract.

16.4 Graphical User Interface

A simple GUI can be added to allow users to upload an image and extract text without using the terminal.

16.5 Better Image Preprocessing

More preprocessing techniques can be added, such as deskewing tilted images, removing shadows, resizing low-quality images, and sharpening text.

16.6 Exporting Results

The extracted text can be exported into different formats such as CSV, Excel, or PDF.

16.7 Combining OCR with Object Detection

A future version can include both text recognition and object detection, where the system can detect objects and extract text from the same image.

17. Conclusion

This project presents a basic but useful image text recognition system using OCR. The system uses Python, OpenCV, and pytesseract to extract text from an image. The image is first preprocessed to improve its quality, then the OCR engine reads the text and converts it into editable digital text.

Through this project, I learned how computer vision can be used to process visual information and how OCR can solve real-world problems related to document digitization. I also learned the importance of preprocessing in improving OCR accuracy.

Overall, the project meets its objective by successfully building a simple OCR pipeline that reads text from an image and saves the final result in an organized way.

18. References / Libraries

- Python
- OpenCV
- pytesseract
- Tesseract OCR Engine
- Pillow
- Matplotlib